

Building a Human-Approved LinkedIn Content Pipeline

A two-workflow n8n system: one workflow learns writing patterns from strong reference posts, and the other turns raw source material (screenshots, PDFs, links and notes) into fact-checked LinkedIn drafts, with a person approving every post before it goes live.

At a glance

Project	AI-assisted LinkedIn content automation
Type	Workflow automation and applied LLM engineering
Orchestration	n8n. Two workflows connected through one vector index: reference ingestion (21 nodes) and the main content pipeline (59 nodes).
AI nodes	Ingestion, 8 of 21: 1 agent, 1 LLM chain, 2 chat-model nodes, 1 structured-output parser, 1 embeddings node, 1 vector store and 1 OCR node Main pipeline, 21 of 59: 5 agents, 1 LLM chain, 8 chat-model nodes, 4 structured-output parsers, 1 embeddings node, 1 vector store and 1 OCR node
Models	Groq-hosted open-weight models (gpt-oss-120b, gpt-oss-20b) with Google Gemini for pattern extraction, strategy, fallbacks and embeddings
Data layer	Pinecone (vector store), Supabase (Postgres)
Integrations	Google Drive, Slack, Gmail, LinkedIn
Status	Reference-content ingestion complete and tested end-to-end. Main pipeline fully built out as nodes, with wiring, configuration and end-to-end testing remaining.

1. Background

The project began as a retrieval-augmented generation (RAG) experiment: answering questions from a company datasheet using a vector database, with Pinecone's managed free tier as the store.

It grew into a two-workflow system for LinkedIn content:

Workflow	Role
Reference-content ingestion	Takes strong reference posts, extracts reusable patterns (hooks, structures, tones, CTAs, anti-patterns) and stores them in Pinecone, behind a deduplication gate.
Main content pipeline	Picks a topic, analyses source material, plans and writes a post, checks it, and routes it through human approval.

The key shift was repurposing retrieval. Instead of retrieving **facts** to answer questions, the vector store holds **patterns** that guide *how* a post is written, while facts come only from the source material. The ingestion workflow fills that store and the main pipeline reads from it, so this case study describes them together as one system.

2. The problem

Posting consistently on LinkedIn involves several separate jobs: deciding what to post, gathering material, shaping it into a post, and reviewing it. Handing all of that to a single LLM prompt tends to cause four problems:

- **Invented specifics.** Models fill gaps with plausible-sounding statistics and details.
- **Privacy leaks.** Source material can contain client names, unreleased information or credentials.
- **Generic output.** Without reference points, posts drift toward the same bland hooks and structures.
- **Loss of control.** Someone has to stay accountable for what is published under their name.

Each AI node in this system exists to answer one of these problems. That is why it uses several narrow AI nodes instead of one large prompt.

3. Design principles

- **Grounded.** A post may only contain claims that trace back to facts extracted from the source.
- **Patterns, not facts, from RAG.** Retrieval informs structure and tone but can never inject unverified claims.
- **Human final say.** Nothing is published without an explicit approval.
- **Auditable.** Every stage leaves a record that can be analysed later.
- **Resilient.** Model fallbacks, and reminders when a reviewer does not respond.
- **Format-agnostic.** Screenshots, PDFs, web pages and pasted text all feed one pipeline.
- **AI where judgement is needed, rules everywhere else.** LLMs handle selection, extraction, planning, writing and review. Routing, logging and publishing stay deterministic.

4. Architecture

The system is two workflows with one point of contact, the Pinecone index. AI nodes are highlighted in blue in every diagram.

4.1 System overview

The ingestion workflow watches a reference-content Drive folder and writes patterns into a Pinecone index. The main pipeline's Content Strategist queries that same index as a retrieval tool. Neither workflow calls the other, so the index is the only interface between them. Both use Gemini embeddings, because a query vector is only comparable to stored vectors from the same model.

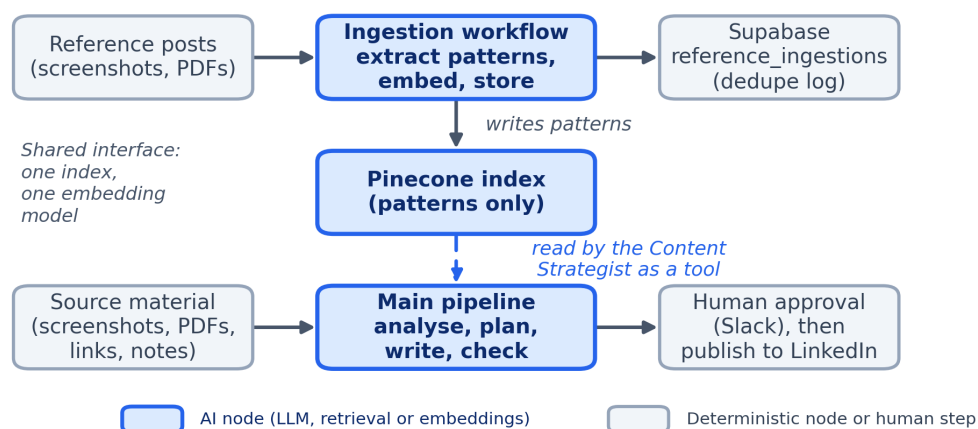


Figure 1. The two workflows share one Pinecone index, which holds patterns only. The extraction prompt forbids repeating a reference post's facts, names or numbers.

4.2 Ingestion workflow

There is one entry point: a Google Drive trigger that fires when a file lands in the reference-content folder. Screenshots and PDFs take different extraction paths, then converge on one text field for pattern extraction.

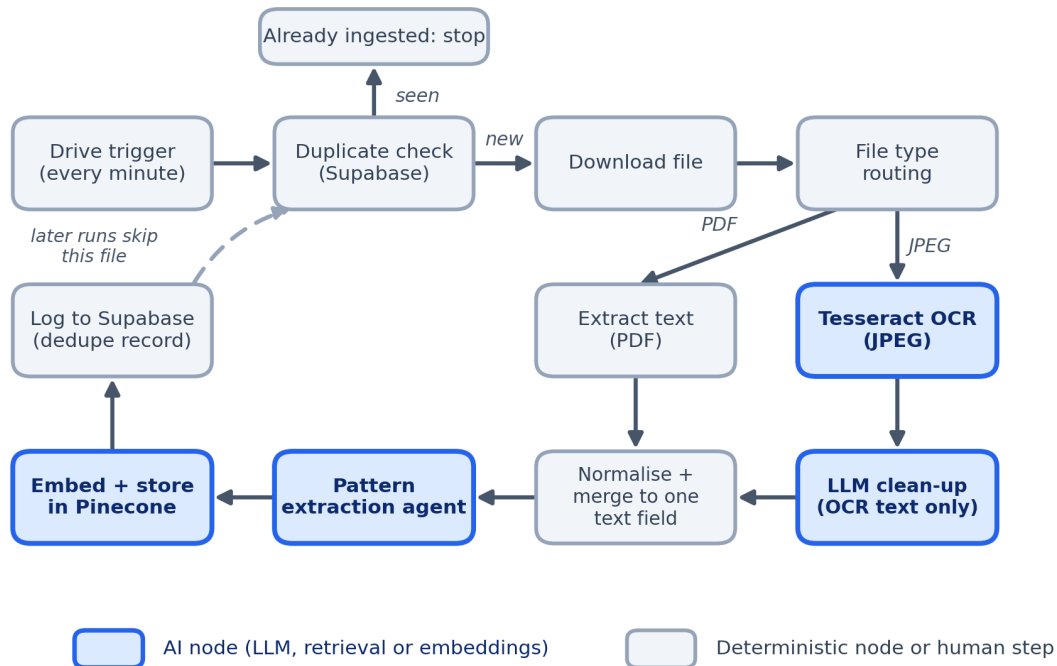


Figure 2. Ingestion workflow. PDFs skip the LLM clean-up; only OCR text passes through it. The Supabase log is written last, so a run that fails midway is never recorded as done.

4.3 Main pipeline

There are two entry points. The **scheduled path** picks a topic from an inventory each morning. The **event path** starts when a file lands in a watched Drive folder and skips topic selection, going straight to extraction.

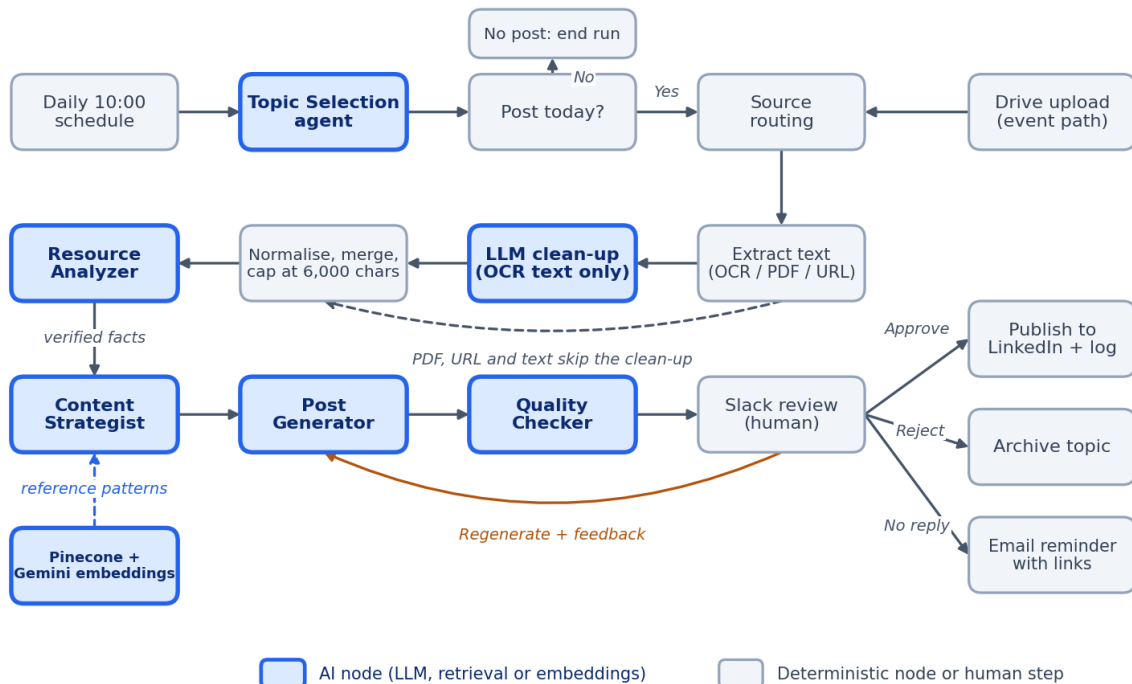


Figure 3. Main pipeline. When nobody answers in Slack, the emailed Approve / Reject links route the run to publishing or archiving. Routing after review is handled by Switch nodes, not by an LLM.

4.4 AI node map

Every AI node, the model behind it, what it produces, and what the system loses without it.

Ingestion workflow

AI node	Model	Produces	Why it is essential
OCR + Clean Data chain	Tesseract, then Groq gpt-oss-20b	Clean text from screenshots	Scrambled line breaks and misread characters would distort the patterns extracted from a post, especially its formatting.
Content Extracter agent	Gemini	One pattern record per post: hook, structure, tone, formatting, CTA, anti-patterns	Without it the index would store raw posts, and retrieval would carry other people's facts and wording into new drafts.
Structured Output Parser (1)	Schema enforcement	Validated JSON with six required fields	The embed step reads fields such as <code>output.hook_type</code> . Free text would break that handoff.
Gemini embeddings + Pinecone vector store	Gemini embeddings, Pinecone	Searchable pattern vectors with source metadata	Makes the patterns searchable by meaning, which is what the main pipeline's strategist relies on.

Main pipeline

AI node	Model	Produces	Why it is essential
Topic Selection agent	Groq gpt-oss-120b	Chosen topic id and reasoning, or a no-post call	Without it the pipeline could only pick by a fixed rule and would post every day, whether or not a topic fits.
OCR + Clean Data chain	Tesseract, then Groq gpt-oss-20b	Clean text from screenshots	OCR mistakes would flow into the facts list and be treated as verified.
Resource Analyzer agent	Groq gpt-oss-120b, Gemini fallback	Verified facts, angles, technical details, privacy flags	No closed set of facts to hold later stages to, and no privacy pre-screen.
Content Strategist agent	Gemini	Strategy contract (angle, format, hook, structure, CTA, length)	The generator would improvise, and the checker would have no plan to compare the draft against.
Pinecone vector store + embeddings	Pinecone, Gemini embeddings	Reference patterns, retrieved as a tool (index filled by the ingestion workflow)	The strategist would fall back on generic model knowledge and posts would drift toward generic hooks.
Post Generator agent	Groq gpt-oss-120b, Gemini fallback	The draft post	It is the only node that turns a strategy and facts into copy.
Quality Checker agent	Groq gpt-oss-120b	Structured verdict on facts, strategy, readability, privacy	The generator would be grading its own work, and the reviewer would see no flagged evidence.
Structured Output Parsers (4)	Schema enforcement	Validated JSON for four agents	Later nodes read fields such as <code>output.selected_topic_id</code> . Free text would break those handoffs.
Fallback models (2)	Gemini	Continuity for the analyzer and generator	An outage or rate limit at either stage would stall the whole run.

5. Ingestion workflow, stage by stage

Each stage below lists what happens and, for every AI node, a spotlight explaining why that node is essential. Stages without an AI node are deliberately deterministic.

Stage 1: Trigger and duplicate check

A Google Drive trigger polls the reference-content folder every minute and fires on each new file. Before anything is downloaded, a Supabase lookup checks the `reference_ingestions` table for that Drive file ID, and a filter stops the run if a row already exists. Supabase returns one empty object when nothing matches, so the filter passes only items with an empty ID. New files are downloaded and a Switch node routes them by MIME type: PDF to text extraction, JPEG to OCR.

No AI node is involved. The check runs before any download, OCR or LLM call, so a repeat file costs almost nothing.

Stage 2: Extraction and normalisation

Each source type gets its own path, and both converge on a single `text` field.

Source	Handling
Screenshot (JPEG)	Tesseract OCR, then an LLM clean-up pass (AI node, below)
PDF	Text extraction, no LLM pass needed

Both paths are normalised into the same field and merged into one stream for the AI stage.

AI NODE OCR + Clean Data chain	
MODEL	Tesseract for OCR, then Groq gpt-oss-20b in a basic LLM chain
OUTPUT	Plain text (no parser needed)
WHAT IT DOES	Tesseract reads the text off a screenshot. The LLM then reconstructs broken line breaks and misread characters, and strips like and comment counts, under an explicit instruction not to add or remove content.
WHY IT'S ESSENTIAL	Pattern extraction judges how a post is laid out, including its line-break rhythm and paragraph shape. OCR output with scrambled breaks or garbled symbols would produce wrong formatting patterns, and engagement counts would be read as part of the post. A smaller model is enough for this mechanical repair job.
DESIGN NOTE	Only the screenshot path uses an LLM. PDFs are read with direct text extraction, so AI is used only where rules cannot do the job.

Stage 3: Pattern extraction

AI NODE Content Extracter agent

MODEL	Gemini chat model
OUTPUT	Structured JSON: <code>hook_type</code> , <code>structure</code> , <code>tone</code> , <code>formatting_notes</code> , <code>cta_type</code> , <code>anti_patterns</code>
WHAT IT DOES	Analyses a reference post for how it is written, not what it says: how the first line grabs attention, the overall shape, the voice, the visual formatting, the kind of call to action, and anything that likely hurt engagement.
WHY IT'S ESSENTIAL	This node is what makes the vector store hold patterns instead of content. Without it the index would fill with copies of other people's posts, and retrieval could leak their facts and wording into new drafts. It also turns a messy post into six comparable fields the strategist can reason about.
GUARDRAILS	The prompt forbids repeating the post's specific facts, names, numbers or claims. All six fields are required by the output schema, and <code>anti_patterns</code> may be an empty list, so the agent is not pushed to invent flaws.

Stage 4: Embed, store and log

The six pattern fields are flattened into one text blob and split with a recursive character splitter (1,500 characters, 200 overlap). Each chunk is embedded with Gemini embeddings and inserted into the `linkedin-reference-content` Pinecone index with three metadata fields: the source file ID, the ingestion timestamp and the hook type. Only then is a row written to `reference_ingestions`, so the file is skipped on future runs and a run that fails partway is never recorded as done.

AI NODE Gemini embeddings + Pinecone vector store

MODEL	Gemini embeddings, with Pinecone as the vector store in insert mode
OUTPUT	Pattern vectors with <code>source_file_id</code> , <code>ingested_at</code> and <code>hook_type</code> metadata
WHAT IT DOES	Converts each pattern record into a vector and stores it, so the main pipeline can find patterns by meaning rather than by keyword.
WHY IT'S ESSENTIAL	This is the bridge between the two workflows. The Content Strategist's retrieval tool searches this index, so the quality and consistency of what is stored here sets the ceiling for how well posts are shaped.
DESIGN NOTE	The main pipeline must embed its queries with the same model used here, or the similarity scores are meaningless. The source-file metadata ties every vector back to its Drive file and its log row.

6. Main pipeline, stage by stage

Each stage below lists what happens and, for every AI node, a spotlight explaining why that node is essential. Stages without an AI node are deliberately deterministic.

Stage 1: Triggers

A schedule fires daily at 10:00 and tags the run `scheduled`. A Google Drive trigger polls a Post-content folder every minute and tags new-file runs `event`. No AI node is involved.

Stage 2: Topic selection

Topics live in a Supabase inventory, exposed through an `eligible_trending_topics` view that applies cooldown rules so recent topics do not repeat. A filter node strips the empty placeholder object Supabase returns when nothing matches, so the agent never sees a phantom topic.

AI NODE Topic Selection agent

MODEL	Groq gpt-oss-120b
OUTPUT	Structured JSON: <code>selected_topic_id</code> , <code>reasoning</code> , <code>no_post_recommended</code>
WHAT IT DOES	Receives the full list of eligible topics as JSON and picks the single best one to post about today, or recommends skipping.
WHY IT'S ESSENTIAL	A queue can be ordered but not judged. This node turns a list of possible topics into a decision, and it is the only place the pipeline can conclude that nothing is worth posting. Without it the workflow would fall back to a mechanical rule such as oldest-first and publish daily regardless of fit.
GUARDRAIL	The <code>reasoning</code> field is required, so every choice is explainable in the logs. <code>no_post_recommended</code> is a first-class output rather than a forced pick.

Stage 3: Extraction and normalisation

Each source type gets its own path, and all four converge on a single `text` field.

Source	Handling
Screenshot (PNG)	Tesseract OCR, then an LLM clean-up pass (AI node, below)
PDF	Text extraction
URL	HTTP fetch, HTML extraction that skips navigation, forms and other page chrome, then a code step that removes generic site noise (cookie banners, newsletter prompts, share buttons)
Pasted text	Passed straight through

The merged text is capped at 6,000 characters to keep prompts within a predictable token budget.

AI NODE OCR + Clean Data chain

MODEL	Tesseract for OCR, then Groq gpt-oss-20b in a basic LLM chain
OUTPUT	Plain text (no parser needed)
WHAT IT DOES	Tesseract reads the text off a screenshot. The LLM then repairs broken line breaks and misread characters and strips like and comment counts, under an explicit instruction not to add or remove content.
WHY IT'S ESSENTIAL	Everything downstream trusts the extracted text. A misread digit or a merged word in OCR output would be extracted by the analyzer as a verified fact. Cleaning at the source protects the facts-only guarantee. A smaller model is enough for this mechanical repair job.
DESIGN NOTE	Only the screenshot path uses an LLM. PDFs, URLs and pasted text are cleaned with deterministic rules, so AI is used only where rules cannot do the job.

Stage 4: Resource analysis

AI NODE Resource Analyzer agent

MODEL	Groq gpt-oss-120b, with a Gemini fallback model attached
OUTPUT	Structured JSON: <code>facts[], topic, angles[], technical_details, privacy_flags[]</code>
WHAT IT DOES	Reads the truncated source text together with the topic and source type, and extracts only what is explicitly stated, with no inference. It also lists possible content angles and flags anything involving client names, unreleased information or credentials.
WHY IT'S ESSENTIAL	This is the trust boundary of the pipeline. It converts unstructured text into a closed list of facts that the strategist, generator and checker are all held to, and it screens for sensitive details before anything is written. Every later stage depends on it, which is why it is one of only two agents with a fallback model.
GUARDRAIL	The prompt limits facts to what is explicitly stated. The <code>privacy_flags</code> field is required in the output schema.

Stage 5: Content strategy

AI NODE Content Strategist agent + Pinecone + embeddings

MODEL	Gemini chat model, with the Pinecone index attached as a retrieval tool and Gemini embeddings for the queries
OUTPUT	Structured JSON: <code>angle, format, hook_type, structure, cta_type, target_length, reasoning</code>
WHAT IT DOES	Chooses one angle, format, hook, structure, CTA and length for the post. Before deciding, it queries the reference-pattern index, which the ingestion workflow fills, for patterns that fit the topic and angle. The resulting contract is logged to <code>strategy_contracts</code> .
WHY IT'S ESSENTIAL	It separates deciding how to write from writing. Without a strategist the generator would improvise, and the checker would have nothing to measure the draft against. The Pinecone tool stops the strategist defaulting to generic LinkedIn advice by grounding its choice in patterns from strong reference posts. The embeddings node turns each query into a vector so that semantically similar patterns are found, and it must use the same embedding model as the ingestion workflow or the similarity scores are meaningless.
GUARDRAILS	The prompt forbids structures that need data absent from the verified facts, asks the agent to say so when the facts are too thin, and tells it not to assume any performance history.

Stage 6: Post generation and quality check

AI NODE Post Generator agent

MODEL	Groq gpt-oss-120b, with a Gemini fallback model attached
OUTPUT	Free text. There is no parser because the output is the post itself.
WHAT IT DOES	Executes the strategy contract exactly, using only the verified facts. The same node handles first drafts and regenerations: when reviewer feedback exists, it is placed at the top of the prompt and the agent is told to address it specifically.
WHY IT'S ESSENTIAL	It is the only node that writes copy. Its constraints are what turn a general-purpose model into a controlled writer: no invented facts, statistics or client names, at most three hashtags, no emojis, and a closing CTA that matches the contract. Handling regeneration in the same node keeps all writing rules in one place.
GUARDRAIL	When a structural beat calls for a statistic the facts do not contain, the prompt tells the agent to phrase it qualitatively or skip it instead of inventing a figure.

AI NODE Quality Checker agent

MODEL	Groq gpt-oss-120b
OUTPUT	Structured JSON: <code>fact_check_pass</code> , <code>unsupported_claims[]</code> , <code>strategy_compliance_pass</code> , <code>strategy_deviations[]</code> , <code>readability_notes</code> , <code>privacy_security_flags[]</code> , <code>overall_summary</code>
WHAT IT DOES	Evaluates the draft against the verified facts and the strategy contract on four checks: factual support, strategy compliance, readability and repetition, and privacy and security. It cites the exact phrase for every issue and catches subtly altered facts such as a rounded-up statistic or a dropped qualifier. It reports only and never rewrites.
WHY IT'S ESSENTIAL	A generator grading its own work is the weakest form of quality control. This node is an independent pass with a different job: adversarial and evidence-based. It is what puts concrete red flags in front of the human reviewer in Slack. Because it never rewrites, it cannot quietly introduce claims of its own.
DESIGN NOTE	It currently runs on the same model as the generator, so its independence comes from its role, its prompt and the inputs it receives.

Stage 7: Human review in Slack

A Slack message carries the draft, the quality summary, security flags and any unsupported claims, with a form offering Approve, Reject or Regenerate plus a feedback field. Every decision is logged to `post_review_log`.

No AI node here, by design. Approval is a person's click, and the routing that follows is done by deterministic Switch nodes. Keeping an LLM out of the approval path makes the outcome predictable and auditable.

Stage 8: Routing

Decision	What happens
Approve	Post is published to LinkedIn
Reject	Topic is archived in the inventory
Regenerate	Reviewer feedback is attached to the strategy and sent back to the Post Generator, which is told to address it specifically
No response	An email with Approve / Reject links goes out and the workflow waits for a click, then routes as above

Stage 9: Publish and log

Published posts are written to `published_posts` along with the strategy fields that produced them (angle, format, hook type, structure, CTA type, target length), so future analysis can connect how a post was built to how it performed.

Across stages: structured output parsers and fallback models

AI SUPPORT NODES Structured Output Parsers (4) and fallback models (2)

WHAT THEY DO	Four parsers enforce a JSON schema, with required fields, on the Topic Selection, Resource Analyzer, Content Strategist and Quality Checker agents. Two Gemini fallback models are attached to the Resource Analyzer and the Post Generator.
WHY THEY'RE ESSENTIAL	Agents hand work to each other through expressions such as <code>\$json.output.angle</code> . Free-form text would make every handoff fragile, while a schema turns each handoff into a data contract that fails loudly. The fallbacks sit on the two stages the rest of the run cannot proceed without: the fact base and the draft.

7. Key design decisions

Decision	Why	Trade-off
Separate specialist agents with structured outputs	Each step has one job and a defined schema, and can be tested or swapped independently	More nodes, tokens and latency than one big prompt
A strategy contract between strategist and generator	The generator executes a plan instead of improvising, and deviations become detectable	Less creative freedom for the generator
Facts-only grounding, with a qualitative fallback	When the facts cannot support a statistic, the draft is told to write a qualitative line or skip the beat, never invent a figure	Posts can be more modest when source material is thin
RAG for style patterns, not facts	Retrieval can shape hooks and structure but cannot smuggle in unverified claims	Pattern quality depends on the reference posts ingested
Extract patterns, never content	The extraction prompt forbids repeating a post's facts, names or numbers, so the index can shape style without carrying claims	Extraction quality depends on how well the model reads a post
Duplicate check before any processing	A lookup on the Drive file ID skips repeat files before download, OCR or any LLM call	File-level only: a re-uploaded copy with a new file ID counts as new
One embedding model across both workflows	Stored and query vectors stay comparable, so similarity scores mean something	Changing the model means re-embedding the whole index
Independent QA agent that never rewrites	Keeps the reviewer honest and gives the human concrete evidence instead of a silent edit	A failed check still needs a human or a regeneration to fix it
Model choice by task	The larger model handles judgement-heavy agents, a smaller one handles mechanical OCR repair, and Gemini handles pattern extraction, strategy and retrieval	More providers and credentials to manage
Fallbacks on the two critical agents	The analyzer and generator keep working if the primary model or provider fails	Fallback models may behave slightly differently
Human approval with a feedback loop	Keeps a person accountable, and feedback flows into the next attempt	Adds latency, softened by timeout reminders
Supabase audit tables for every stage	Creates the dataset needed to learn what works	Extra writes and schema to maintain

8. Challenges and how they were handled

Noisy inputs. OCR output and scraped web pages are messy. The system uses two stages of cleaning: structural extraction and targeted noise removal for web pages, and an LLM repair pass for OCR. The analyzer and the pattern extractor therefore see readable text.

Keeping facts out of the style library. A reference post is full of its own specifics. If those reached the index they could resurface in new drafts. The extraction prompt forbids repeating them, and the stored record has fixed fields for structure, tone and formatting only.

Two workflows, one index. Patterns are stored by one workflow and searched by another, so both must embed with the same model. Both use Gemini embeddings, and the requirement is treated as a hard constraint rather than a preference.

Processing the same reference twice. Re-ingesting a post would duplicate its patterns in the index and skew retrieval toward it. The workflow looks the file up before downloading it and logs it only after the upsert succeeds.

Models asking for data that does not exist. A strategist can design a post structure that demands a statistic the source never provided. The fix was applied at both ends: the strategist is told not to design structures that require absent data, and the generator is told to handle such beats qualitatively or skip them.

Awkward database behaviour. Supabase returns one empty object instead of zero rows when a query matches nothing, which would look like a real record downstream. Dedicated filter nodes handle it, both for the duplicate check and for the topic inventory that feeds the Topic Selection agent.

Waiting on a human. Approval flows stall if nobody answers. Timed waits plus an email fallback with one-click links keep runs from hanging silently.

A cold start with no history. No performance data exists yet, so the strategist is explicitly told not to reference or assume any. Decisions rest on the analysis and retrieved reference patterns.

9. Current status

Area	Status
Reference-content ingestion	Complete, tested end-to-end
Main pipeline	Every stage from topic selection to publish and logging exists as nodes, including all AI nodes described above
Topic inventory and eligibility view	Designed; needs to be created and populated with real rows
Duplicate-topic check	Designed as a reselection loop capped at three attempts, checking against a published-posts vector index. The index still needs building and the loop testing against real duplicates.
Cooldown reset	Decided: the publish step will record <code>last_posted_at</code> . Not yet implemented.
LinkedIn publishing step	Node in place; author and post-text configuration remaining
End-to-end run on real data	Pending

10. Results and how success will be measured

There are no production numbers yet, so none are claimed here. The system is instrumented so the following can be measured from its own logs once it runs on real topics:

- Approval, rejection and regeneration rates from `post_review_log`
- Unsupported-claim and privacy flags per draft from the Quality Checker output

- Which hook types, structures and CTAs get approved from `strategy_contracts` and `published_posts`
- Time from source material to approved draft
- Size and hook-type mix of the reference library, from `reference_ingestions` and the index metadata

11. What's next

- Finish the topic inventory, eligibility view and duplicate-check loop
- Compare strategy choices against approval outcomes using `post_review_log` and `published_posts`
- Idea: run the Quality Checker on a different model family from the Post Generator, to reduce shared blind spots between writer and reviewer

Skills demonstrated

Multi-agent LLM orchestration, retrieval-augmented generation, ingestion pipelines with deduplication and embeddings, prompt design for grounding and privacy, structured outputs, model selection and fallback design, human-in-the-loop approval flows, data modelling for auditability, and integration across Google Drive, Slack, Gmail, LinkedIn, Pinecone and Supabase.